Editor's Comments

Thanks for your submission to the SJ. We've heard back from an expert reviewer – please see the file attached. You'll see that the referee is broadly supportive, but has some "concerns and recommendations" for you to address. I have looked at your materials too and have the following comments (not necessarily in order of importance):

> We appreciate the chance to revise and improve on the program. Both the reviewer and you as Editor made a number of excellent suggestions for improving `wxsum`. We have adopted nearly every one and added a few new options on our own. We have also added a small package, `wxprep`, that does pre-processing to make use of `wxsum` easier. Since we were adding so many features, we took the opportunity to do a nearly complete rewrite of the code using Mata to do the calculations (a change we had long wanted to make but had never taken the time to do). As such, the codebase looks very different than in the previous submission. Using Mata makes the code much more efficient in both the number of lines in the code and in the time the package takes to do the calculations.
>
> For transparency, we want to acknowledge our use of Claude Code and Google's Antigravity IDE integration with Gemini. None of the authors are as conversant in the Mata language as we are in the Stata language, so we relied on Claude and Gemini to do the Mata coding. To ensure accuracy of the code the AIs wrote, we set up an algorithmic battleground in which each AI completely independently wrote code which we then ran on the same data to compare outcomes. After this initial, independent development, we had each AI review the other's code and make suggestions for improvements, iterating in this way a half dozen times. Finally, we created an audit runner with 165 different tests, both to check for bugs/failures but also to compare output to a "hand calculated" oracle. We rewrote and refined the code until it passed all tests. The audit confirms the package runs without bugs and does the calculations correctly.
>
> We are happy to provide the code audit to you and the reviewer or make it part of the final submitted package. We are also happy to provide whatever sort of AI-use statement that the editors of *Stata Journal* would like.

1. I support the referee's general remarks, and also the requests for greater functionality.

   > We have adopted all but one of the referee's suggestions (deviations in early- and late-season spells) and have provided our reasoning for not calculating these statistics.

2. To me, you've provided useful functionality but it's fairly narrow in terms of constraints on what is required in for inputs and the nature of the outputs. A devil's advocate might argue that -wxsum- in its current form is really helpful for your group and those like you, but not for a wider SJ community of users of climate/weather data. Naturally, the SJ has the interests of the latter at heart. So, I think you take a two-pronged strategy in revisions: (i) make a stronger case that the current functionality is actually of widespread utility (i.e., even if the focus is narrow, there are lots of researchers who could use the functionality); (ii) help the wider Stata-using community more.

> We have endeavored to make the case for the wide usefulness of the package following the two prongs you suggest. We have added a substantial amount of new functionality so that the package produces many more statistics over a variety of windows and ranges. These include all the improvements suggested by you and the reviewer. The package now calculates more statistics, allows for different windows, and different baselines. These options expand the domains where `wxsum` can be used. We have also written a companion package, `wxprep` that accepts both long and wide data and does pre-processing so that the output is in exactly the format needed by `wxsum`. This relaxes the constraints of the type of data that can be used.
>
> We also now document the demand for such a package based on a search of questions asked on Statalist. Requests for guidance in manipulating weather panel data and calculating specific statistics are not unusual. Examples include:
>
> - How to create degrees days by id and year using daily temperature
> - Construct annual rainfall panel data (across countries and years)
> - Creating dummy variable equal to one if current mean is one standard deviation above/below the historical mean
> - Code for Approximating degree days with monthly average temperature
> - Generate variables to capture heat stress

3. Relatedly, the nature of the variables used and provided seem rather field-specific, specifically, ag and ag econ focused. I am more familiar with the use of climate/weather data in the biosocial field (including child development). See, e.g., the discussion of a wide(r) variety of heat metrics in the the Plos-One article at https://doi.org/10.1371/journal.pone.0330498 (disclosure: lead author is one of my PhD students, now graduated). (Compared to your variables, there are metrics related to e.g. wet bulb temperature, and humidity, etc.) This raises the question of how to deal with variables summarising metrics at the input stage that (a) differ from your example sets (to what extent are they hard-wired?), and possibly (b) combine heat and rainfall data to produce 'composite' metrics? Also, the nature of a "season" sounds ag-specific. Could one, for instance, use other period definitions such as months of pregnancy length (start and end dates are not tied to calendar time). Is there a way for users to produce differently-defined output variables (everything seems quite hard-wired at present)?

> We trust that the new options we have added to the package goes a long way in addressing this comment. We now include fixed baselines, we include unit-specific start and end dates for defining the window, as well as unit-specific event dates. We provide alternative ways to construct bins and the package is now `r(class)` and produces a number of useful "meta" metrics for later use. We have also added a pre-processing package to lower the initial burden on the user.
>
> We have chosen not to include calculation of higher-order or composite metrics like SPEI or wet bulb temp. The primary reason is that these require data on not just rainfall and

> temperature but things like humidity, soil moisture, wind speed, etc. We felt that users who would take the time to collect data from a variety of different sources for input into these composites would likely already have their own routine for calculating them. the majority of researchers who would use composite metrics would grab the measure already calculated and included in data sets like MERRA-2 or ERA5. So, the number of users who would take advantage of a routine to calculate wet bulb temp in `wxsum` is likely very small.
>
> In terms of using composite metrics as **inputs** to `wxsum`, this is already possible. The package is agnostic to what the input data actually is and what the unit of measure is. There is nothing to stop a user running `wxsum` on composite metrics. In these cases, not all output statistics will be meaningful but the user can drop those. In the help file we now mention the ability use of other types of data with the package.

4. Relatedly, the referee's remarks are pertinent here, including provision of a companion command to help prepare data for the tightly-constrained input format you require (both in terms of wide format, format of variable names, etc.)

   > We have produced a companion command, `wxprep`, to help prepare the data. The help file and journal article describe its functionality and useful options.

5. Relatedly, I think you need to add functionality ("post-calculation" commands) or otherwise provide examples so that users can find out and check the nature of the datasets created by -wxsum-. Things that come to my mind from playing with your code include: the number of variables created, the number of separate units identified by some ID variable, patterns of missing, and also save for the record some or all of other choices such as start and end dates. If you made your command r(class), then it would be useful to save some of this stuff in r() results that could be later retrieved (cf. -return list-). You could also have the program pump out some of the stuff to the output window once the command has finished.

   > We have converted `wxsum` to an `r(class)` command. It now stores things like the number of observations, the number of variables created, the length of window, and the number of missing observations. The help file provides a complete list. The package also prints out some basic diagnostic information after it runs so that user can quickly and easily see what the package did.

6. Relatedly, you could do more to show your command in action. Your Section 5 "examples" in the text are problematic, not least because you don't show any output or indeed post-calculation use of the created data. I'm not saying you need to provide a full-blown "mini article" for illustrative purposes; what I'm saying is that the examples need extending in a way that better demonstrates the dataset in action in research. You write about merging with other data for analysis. How about finding a small 'toy' dataset you can share, and do some illustrative analysis?

   > We now provide 12 examples using both "real" weather data and simulated data. We did not include all 12 in the article because we felt that would take up too much space. But all 12 are included in the package downloaded from Github or SSC and are discussed in the help files.

7. Relatedly, I would have thought that long form output is the most convenient format for Stata users if they want to inspect the created variables prior to a merge (... and maybe if the to-be-merged data are long as well). In my playing around, I ended up going long to "look at" the rain data because wide was simply far too unwieldy.

   > The package now allows the user to decide if they want output in long or wide format. Long form is the default.

8. The term "killing degree" is used a lot but the label – a strange one! – is not explained.

   > We define KDD in the article and have made the label more useful.

9. In your discussion of Stata capacity, you need to explicitly discuss -set maxvar-. (Note how when I used your rain dta set, Stata complained until I reset maxvar.)

   > One of the first things that the package now checks for is a maxvar value that is sufficiently large to accommodate the data. If not, it produces the following error: "wxsum needs about %f variables, but maxvar is %f. Use shape(long) or keep(), or save your data, type -clear all-, -set maxvar #- with a larger number, reload the data, and run wxsum again." There is also a section in the help file that discuss this and how if one is using Stata/BE they would be limited to about 5 years of data.

10. A related problem: you don't provide any example do-files and the (plain text) log-files generated by them (cf. mine attached). We must have there, at the very least producing everything that you paste into the article text. See the Submission Guidelines.

    > The example .do files and plain text log-files are now included with the submission.

11. Conclusions: this is simply a summary, not providing genuine conclusions, hence risking redundant repetition. We want some genuine forward-looking conclusions. See the discussion of this Section in the Submission Guidelines to understand what we want, and rewrite the section accordingly (with less summarising).

    > We have reviewed the Submission Guidelines and rewritten the Conclusion section accordingly.

12. Remove all -capture program drop- lines in your ado-file. (See https://www.statalist.org/forums/forum/general-stata-discussion/general/1758652-on-capture-program-drop)

    > We have removed all -capture program drop- lines in the code.

13. Your adofiles uses a combination of temporary variables and hard-coded variables (the latter are mostly derived using your -egen- commands, I think – the stuff you refer to in your Calculations section in the text). Your Table 3 and discussion doesn't make clear that the variable names for output are hard-coded, I think. Usually, creation of hard-coded variable names is deprecated by Stata programmers. Instead one would typically do calculations using tempvars and at the end create variables that are clones of the tempvars – and users could optionally choose what to call those variables. A signal advantage of this practice is that if the program stops prior to completion no permanent variables are left behind. Please reflect on what you're doing in your program and, if you maintain your current practice, provide a persuasive justification for doing so explaining why it's not a problem in your case.

> We no longer use the `egen` command nor do we hard-code the variable names as the package runs. Instead, we have moved all calculations to Mata memory and any dataset modifications are guarded by preserve/restore. So, if the command fails its run, the user gets back the original, unmodified dataset with no leftover variables from the partial run.
>
> In terms of allowing the user to name variables, the package creates dozens to hundreds of variables depending on the number of years in the data. Allowing the user to name each one would be impractical. Instead, we now allow users the option to define `outstub()` which adds a custom prefix to the variables. This allows the user to run `wxsum` without running into the problem of variable names already existing.

14. Why not add a -quietly compress- after creating your variables. My playing with your rain data suggests that space can be saved. [Addressing some of the issues mentioned in point #3 above might be mentioned here if it's too hard to address them now?]

    > The package now runs -quietly compress- at the end before it outputs the results.